

UNIVERSIDAD POLITÉCNICA DE MADRID



ESCUELA TÉCNICA SUPERIOR DE INGENIEROS INFORMÁTICOS

MEMORIA DEL PROYECTO DE PROGRAMACIÓN ENSAMBLADOR

Claudia Lastra Díaz
ID grupo : claudialastraa

1. Resumen diario (fechas y horas diarias)

Fecha	Autor	Resumen de Actividad	Tiempo invertido
25/10/2025	Claudia Lastra	Revisión de la documentación del proyecto, repaso de lo realizado el año anterior.	1 ½ horas
26/10/2025	Claudia Lastra	Implementación inicial de las macros y de la primera subrutina LongCad	2 horas
27/10/2025	Claudia Lastra	Implemento subrutinas BuscarCad y Coincidencias, y reviso la documentación de los casos de prueba para comprobar que estas tres subrutinas del Hito1 funcionan de forma correcta.	3 horas
29/10/2025	Claudia Lastra	Revisar documentación para el Hito2, e implementación de PoneBitA1.	1 hora
31/10/2025	Claudia Lastra	Para aprovechar correcciones comienzo a implementar la subrutina Comprime.	2 horas
7/11/2025	Claudia Lastra	Continuo con la subrutina Comprime.	1 ½ horas
11/11/2025	Claudia Lastra	Revisó el código del Hito1 y realizó una entrega	½ horas
15/11/2025	Claudia Lastra	Implementar subrutina LeeBit y comprobar que pasa los casos de prueba, y continuó con las subrutina Comprime	2 horas
18/11/2025	Claudia Lastra	Continuación con la subrutina Comprime con la cual sigo teniendo muchos errores	1 ½ hora
19/11/2025	Claudia Lastra	Continuo con comprime, que continúa sin pasarme los casos de prueba de forma correcta	1 hora
20/11/2025	Claudia Lastra	Comienzo la implementación de la subrutina Descomprime y consigo que me pase los casos de prueba.	3 horas
21/11/2025	Claudia Lastra	Continuo con la subrutina comprime, que continúa sin pasarme los casos de prueba de forma correcta	1 hora
25/11/2025	Claudia Lastra	Continuo con la subrutina Comprime, que continúa sin pasarme los casos de prueba de forma correcta	2 horas

26/11/2025	Claudia Lastra	Continuo intentando averiguar qué le pasa a mi Comprime	2 horas
27/11/2025	Claudia Lastra	Continuo con Comprime	2 horas
30/11/2025	Claudia Lastra	Comienzo Verifica, realizó un único caso de prueba el cual pasa	2 horas
5/12/2025	Claudia Lastra	Decido realizar una entrega para ver si al menos me pasa alguna prueba.	1 hora
7/12/2025	Claudia Lastra	Continuo con Comprime y me doy cuenta del error.	2 horas
11/12/2025	Claudia Lastra	Comprime pasa el primer caso de prueba pero no el segundo así que lo arreglo.	2 horas
12/12/2025	Claudia Lastra	Continuo con verifica para intentar arreglar sus fallos y realizar entrega	3 horas
14/12/2025	Claudia Lastra	Continuo con Verifica y creo que he encontrado el fallo	2 horas
15/12/2025	Claudia Lastra	Realizó parte de la memoria, aunque ya haya ido realizando todos los días y realizó entrega para comprobar si he arreglado Verifica	3 horas
16/12/2025	Claudia Lastra	Tras comprobar los resultados de la subrutina Verifica, continuó realizando la memoria. Creo haber encontrado lo que fallaba en Verifica.	2 horas
17/12/2025	Claudia Lastra	Comprobar fallos de la subrutina Verifica y añadir progreso a la memoria.	1 hora
18/12/2025	Claudia Lastra	Compruebo que la subrutina Verifica ahora si funciona de forma correcta, y añado el progreso en la memoria.	½ horas

Este curso le he dedicado aproximadamente un total de : 45 horas.
No estoy contando las horas dedicadas a la convocatoria ordinaria y extraordinaria del año anterior, puesto que he intentado realizarlo nuevamente desde cero.

2. Resumen de Casos de Prueba

Durante la realización de este proyecto, realice distintos casos de prueba para las distintas subrutinas, estos son ejecutados para comprobar el correcto funcionamiento de las subrutinas de nuestro ensamblador. Para cada subrutina se han probado situaciones representativas, verificando tanto el valor devuelto en el registro r29, como los efectos en memoria y el estado de la pila. Los casos se han documentado siguiendo el formato del simulador(contenido de memoria “antes y después”, parámetros en pila y registros), teniendo en cuenta el direccionamiento por byte y el formato little-endian.

Como punto de partida he utilizado los ejemplos proporcionados por el DATSI principalmente para darme una idea general de sí mis subrutinas funcionaban, pero no constituyen el conjunto completo de validación; por ello he añadido pruebas adicionales para cubrir situaciones que no aparecen en estos ejemplos del departamento. En resumen se destacan explícitamente los casos que difieren de los aportados por el departamento.

- Casos de prueba proporcionado por el departamento:

Caso 1. LongCad (Longitud de cadena):

- Eentrada: "Prueba\0"
- Resultado esperado: r29 = 6 (longitud de la cadena "Prueba\0").
- Objetivo: comprobar el cálculo correcto de longitud para una cadena simple.

Caso 2. LongCad (Longitud de cadena):

- Entrada: " 123456789 123456789 12\0A"
- Resultado esperado: r29 = 22 (longitud hasta el \0)
- Objetivo: verificar que la rutina se detiene en el terminador nulo e ignora caracteres posteriores.

Caso 3. BuscaCar (Búsqueda de carácter en cadena):

- Cadena: "*2345*78*0\0"
- Carácter buscado: '5'
- Resultado esperado: r29 = 5
- Objetivo: comprobar que devuelve correctamente la posición del carácter encontrado.

Caso 4. BuscaCar (Búsqueda de carácter en cadena):

- Cadena: "*2345*78*0\0"
- Carácter buscado: '9'
- Resultado esperado: r29 = -1
- Objetivo: validar el comportamiento cuando el carácter no existe en la cadena.

Caso 5. CoincidenCad (Comparación de cadenas):

- Cadena 1: "1234567890\0"
- Cadena 2: "0123456789\0"
- Resultado esperado: r29 = 0
- Objetivo: comprobar que detecta diferencia desde el primer carácter y marca “no coinciden”.

Caso 6. PoneBitA1 (Activar un bit intermedio en un mapa de bits):

- Mapa inicial: mapa1 = 0x00
- Entrada: r30 → &mapa1, r2 = 4
- Resultado esperado: mapa1 = 0x10 (0001 0000)
- Objetivo: verificar la activación correcta de un bit concreto.

Caso 7. PoneBitA1 (Activar dos bits consecutivos):

- Mapa inicial: mapa1 = 0x00
- 1ª llamada: r2 = 2 → esperado mapa1 = 0x04
- 2ª llamada: r2 = 3 → esperado mapa1 = 0x0C
- Objetivo: comprobar activaciones consecutivas sin perder bits anteriores. En esta prueba lo importante es que vaya acumulando bits activados correctamente tras cada llamada.

Caso 8. LeeBit (leer un único bit en un mapa con varios bits activados):

- Mapa: mapa2 = 0x54 (0101 0100)
- Entrada: r30 → &mapa2, r2 = 4
- Resultado esperado: r29 = 1
- Objetivo: confirmar lectura correcta de un bit en estado 1.

Caso 9. LeeBit (leer dos bits consecutivos):

- Mapa: mapa2 = 0x54
- Lectura 1: r2 = 4 → esperado r29 = 1
- Lectura 2: r2 = 3 → esperado r29 = 0
- Objetivo: validar lectura de bits con valores distintos en posiciones cercanas.

Caso 10. Comprime (texto sin compresión):

- Entrada: r30 → cadena6, r0 → texto original (cadena6)
- Resultado esperado: longitud comprimida = 16 y salida coherente en memoria (por ejemplo, en 0x12100).
- Objetivo: comprobar que la compresión genera una salida válida aunque apenas reduzca tamaño.

Caso 11. Comprime (texto con compresión):

Entrada: r30 → cadena8, r0 → texto original (cadena8)

- Resultado esperado: longitud comprimida = 70 y estructura correcta en memoria.
- Objetivo: validar un caso donde sí hay compresión y la salida debe reflejarlo.

Incidencias detectadas en Comprime:

Durante las primeras ejecuciones se observaron errores en la salida generada:

- La longitud devuelta inicialmente era incorrecta (no coincidía con la esperada).
- La cabecera de la salida comprimida aparecía mal construida.
- Además, el contenido almacenado en la zona 3 no correspondía con lo esperado (datos desalineados o incorrectos), lo que afectaba a la verificación y a la posterior descompresión.

Caso 12. Descomprime (Texto no comprimido):

- Entrada: r30 → cadena9, r0 → texto original (cadena9)
- Resultado esperado: longitud = 20 y copia idéntica del contenido.
- Objetivo: verificar que la rutina funciona cuando el contenido no requiere reconstrucción compleja.

Caso 13. Descomprime (Texto comprimido):

- Entrada: r30 → cadena8, r1 → dirección del texto comprimido
- Resultado esperado: longitud = 68 y contenido descomprimido correcto.
- Objetivo: validar descompresión completa y coherente con el texto original.

Caso 14. Verifica (cadena no comprimible):

- Entrada: r30 → cadena6, r0 → texto original (cadena6)
- Resultado esperado: r29 = 0
- Objetivo: comprobar que un texto válido, aunque no comprimible, pasa la verificación.

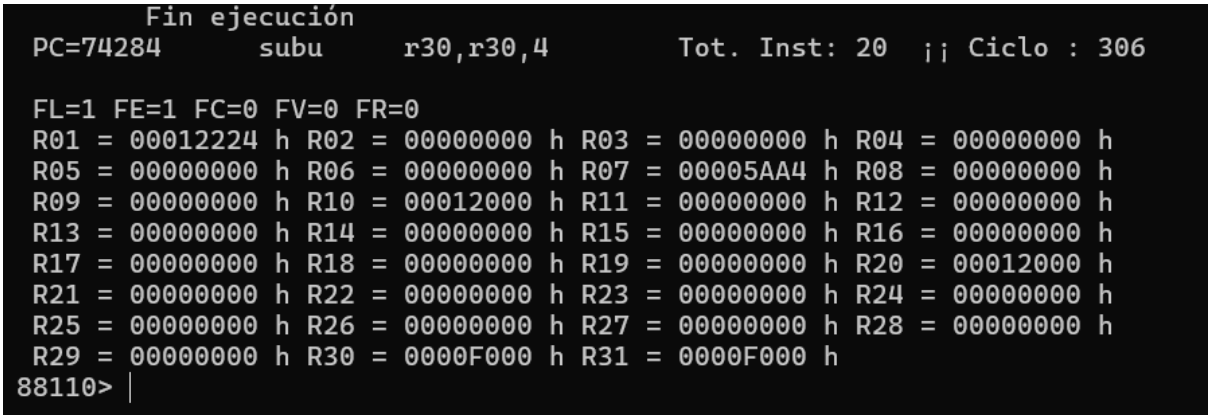
Caso 15. Verifica (texto comprimido):

- Entrada: r30 → cadena8, r0 → texto comprimido (cadena8)
- Resultado esperado: r29 = 0
- Objetivo: confirmar que la verificación acepta un texto comprimido cuando la compresión/descompresión es consistente.

- Casos de prueba adicionales:

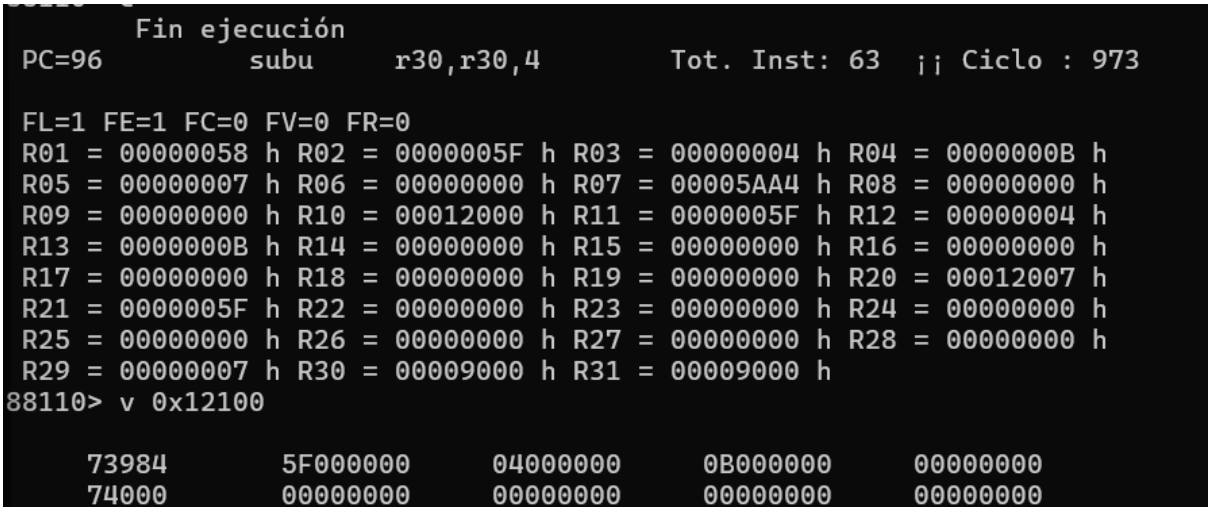
PLongCad:

- Entrada: cadena = “\0” (cadena vacía)
- Esperado r29 = 0 (la longitud no incluye el terminador)



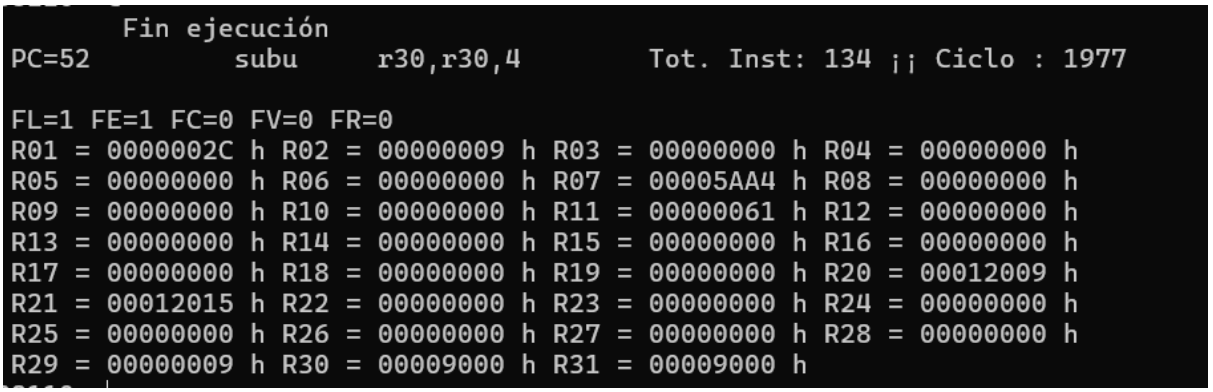
PBuscarCar:

- Entrada: ref = “adc_def_ghi\0”, C = ‘_’, from = 4, to = 11
- Esperado: r29 = 7 (encuentra el ‘_’ en la posición 7 dentro del tramo[from, to])



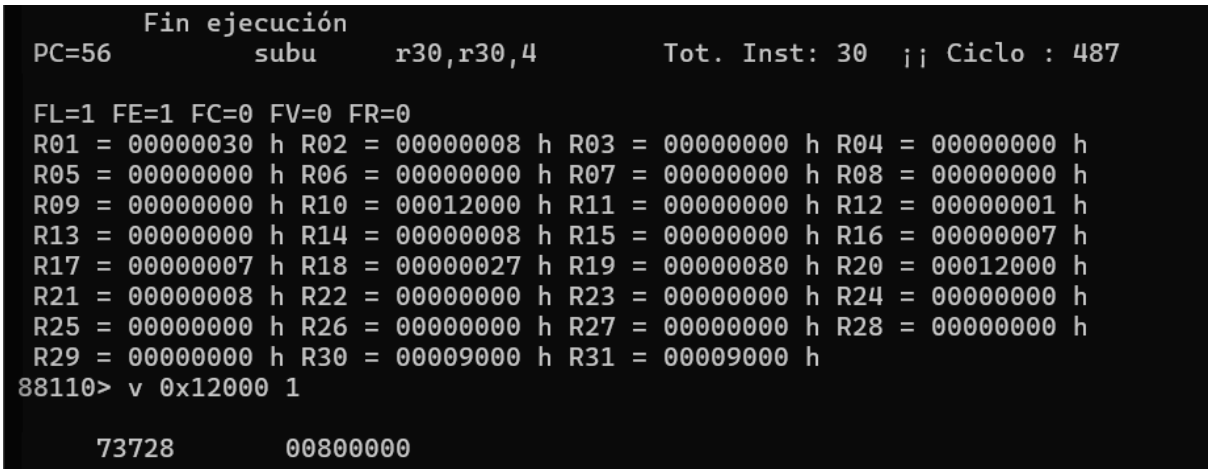
PCoincidenCar :

- Entrada: Cadena3: “estructur\0”, Cadena4: “estructura\0”
- Esperado: r29 = 9 (coinciden e, s, t, r, u, c, t, u, r y se detiene al final de la primera cadena)



PPoneBitA1 :

- Estado inicial: mapa de bits de 2 bytes: [0x00, 0x00]
- Entrada: dirZonaCB -> &mapa, NumBit = 8
- Esperado: mapa final [0x00, 0x80]
  (NumBit = 8 cae en el segundo bytes y activa su bit más significativo)



PLeeBit :

- Mapa: [0x00, 0x80]
- Entrada: dirZonaCb -> &mapa, NumBit = 8
- Esperado: r29 = 1 (ese bit está a 1)

```
       Fin ejecución
PC=56         subu     r30,r30,4        Tot. Inst: 70  ¡¡ Ciclo : 1081

FL=1 FE=1 FC=0 FV=0 FR=0
R01 = 00000030 h R02 = 00000001 h R03 = 00000000 h R04 = 00000000 h
R05 = 00000000 h R06 = 00000000 h R07 = 00005998 h R08 = 00000000 h
R09 = 00000000 h R10 = 00012000 h R11 = 00000000 h R12 = 00000000 h
R13 = 00000000 h R14 = 00000001 h R15 = 00000000 h R16 = 00000000 h
R17 = 00000000 h R18 = 00000000 h R19 = 00000000 h R20 = 00012000 h
R21 = 00000008 h R22 = 00000001 h R23 = 00000000 h R24 = 00000000 h
R25 = 00000000 h R26 = 00000007 h R27 = 00000000 h R28 = 00000000 h
R29 = 00000001 h R30 = 00009000 h R31 = 00009000 h
88110> v 0x12100

    73984       08000000     00000000     00000000     00000000
```

PComprime :

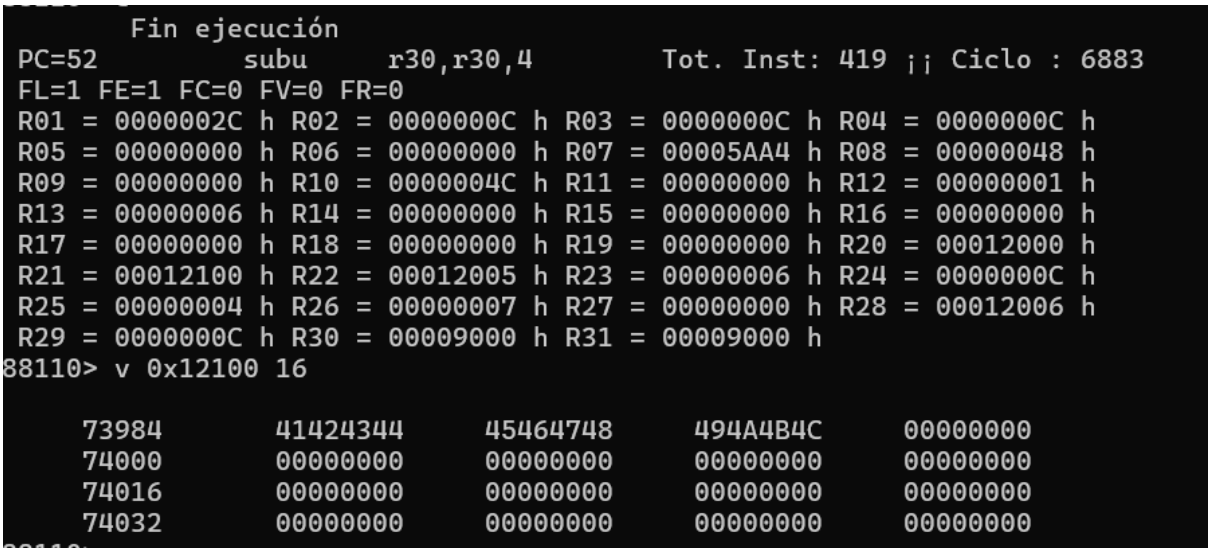
- Entrada: Texto = “ABCDEFGHIJJKLMNO\0” (15 caracteres, sin repeticiones largas)
- Esperado (estructural) :
    - comprdo[0..1] = 0x0F 0x00 (LO=15, little-endian)
    - comprdo[2] = 0x01 (M=1)
    - comprdo[3..4] = 0x06 0x00 (5 + bytes de mapa de bits)
    - Mapa de bits: 1 bytes a 0x00 (no hay subcadenas repetidas de L>=4)
    - Zona3: contiene exactamente ABCDEFGHIJKLMNO (sin \0)
    - Tamaño devuelto : r29 = 5 +1 +15 = 21 bytes (0x15)

```
       Fin ejecución
 PC=52         subu     r30,r30,4        Tot. Inst: 1626        ¡¡ Ciclo : 2
5625
 FL=1 FE=1 FC=0 FV=0 FR=0
 R01 = 0000002C h R02 = 0000000F h R03 = 0000000F h R04 = 0000000E h
 R05 = 00000001 h R06 = 00000004 h R07 = 00005AA4 h R08 = 0000004F h
 R09 = 00000000 h R10 = 00000000 h R11 = 0000000E h R12 = 00009EEC h
 R13 = 00000000 h R14 = 00000000 h R15 = 00000010 h R16 = 00000000 h
 R17 = 0000000F h R18 = 00000000 h R19 = 00000000 h R20 = 00012000 h
 R21 = 00012100 h R22 = 00012115 h R23 = 00000000 h R24 = 00000000 h
 R25 = 00000000 h R26 = 00000006 h R27 = 00000A2B h R28 = 00009EC0 h
 R29 = 00000015 h R30 = 00009F00 h R31 = 00009F00 h
88110> v 0x12100 6

    73984       0F000106     00004142     43444546     4748494A
    74000       4B4C4D4E     4F000000
```

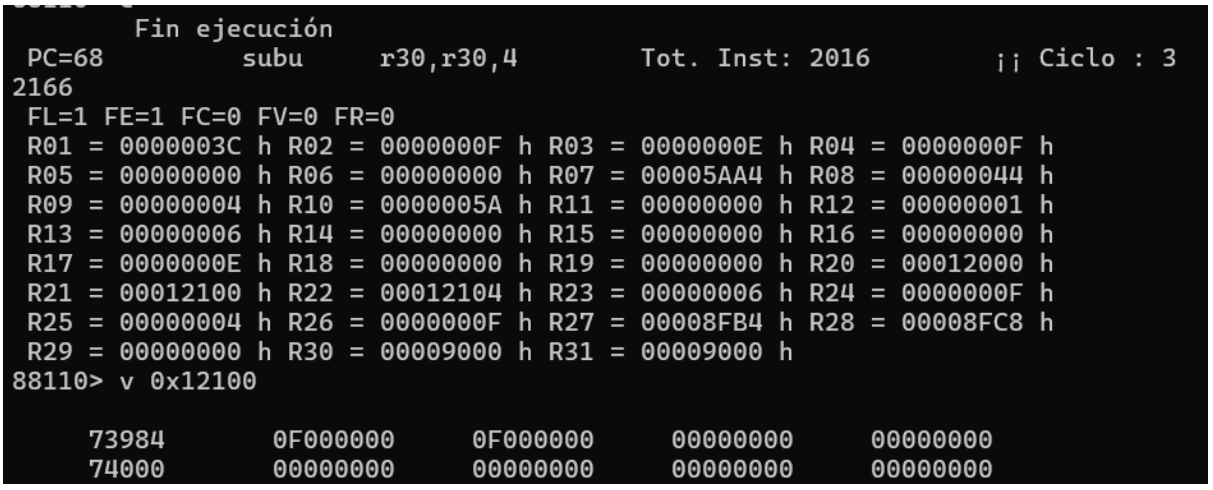
PDescomprime:

- Entrada: com = el comprimido del caso anterior, 0x0601000C, 0x42410000, 0x46454443, 0x4A494847, 0x00004C4B
- Esperado: en desc queda "ABCDEFGHIJKLMNO\0" y r29 = 12 (0x0C) (41424344 45464748 494A4B4C 00000000)



PVerifica:

- Entrada: texto = "ABCDABCDABCDXYZ\0" (fuerza compresión por repeticiones), Long1 y Long2 por dirección
- Esperado: r29 = 0, y además Long1 = 15, Long2 = 15



3. Observaciones finales y comentarios personales

Desde mi punto de vista, el proyecto ha tenido un nivel de dificultad alto, sobre todo por estar desarrollado en ensamblador, donde cualquier detalle puede hacer que algo deje de funcionar sin que sea fácil ver el motivo. Aun así, también ha sido un trabajo muy útil para entender mejor cómo funciona un programa “por dentro” y para ganar paciencia a la hora de probar y corregir errores.

El año pasado ya había intentado este proyecto con otro compañero, pero no logramos que saliera bien. De esa experiencia sí me llevé algunas ideas muy básicas (sobre todo para orientarme al principio), pero este año he preferido hacerlo prácticamente desde cero para no depender de lo anterior y para asegurarme de que entendía cada parte de verdad.

El Hito 1, fueron de las subrutinas que mi compañero y yo realizamos correctamente el año pasado, además que comparándolo con el nivel de ejercicios de exámenes y de ejercicios realizados en el aula, no es de demasiada dificultad, por lo cual no tuve ningún problema en su implementación.

El Hito 2, si que me supuso ligeramente un mayor esfuerzo que el Hito 1, sobretodo LeeBit, no obstante al final conseguí implementarlas de forma correcta.

En cuanto al desarrollo, la parte que más se me atragantó fue Comprime. No solo porque es la rutina más larga, sino porque tiene muchas piezas que tienen que encajar a la vez. Además, estuve bastante tiempo frustrada por un fallo que parecía “invisible”: en mi archivo de pruebas, sin darme cuenta, no había incluido BuscaMax, y eso hacía que los resultados no fueran los esperados. Me estaba desesperando revisando el codigo de Comprime una y otra vez, porque además estaba cambiando cosas que estaban bien, simplemente por que al realizar los casos de prueba todo el rato me daban error puesto que después de llamar a BuscaMax lo que había en r29 realmente seguía siendo lo que había devuelto LongCad, cuando en realidad era una tontería de organización. Cuando lo descubrí, me dio rabia por el tiempo perdido, pero también me sirvió para aprender que a veces el problema no está en el código “difícil”, sino en algo simple que se te pasa por alto.

En cambio, Descomprime fue todo lo contrario: me resultó bastante más directo y fácil de entender. Una vez tenía claro el formato de los datos, ir reconstruyendo el texto original era un proceso mucho más claro, y me permitió comprobar que el trabajo anterior tenía sentido.

Luego en Verifica me ha ocurrido algo parecido a Comprime, no estaba llamando a la subrutina CoincidenCad de forma correcta(una errata muy tonta, aunque me compilaba de forma correcta), y había revisado el código varias veces buscando por un error pero sin darme cuenta, hasta que conseguí darme cuenta y arregle el fallo.

En definitiva, aunque ha sido un proyecto exigente y en algunos momentos desesperante, me quedo con la sensación de haber aprendido mucho y de haber mejorado tanto en la parte técnica como en la forma de enfrentarme a los errores.

Yo, Claudia Lastra Díaz, declaro conocer y manifiesto mi conformidad con las siguientes normas sobre copia y fraude académico:

- el uso para el desarrollo del proyecto de cualquier material de otros grupos/alumnos o generado automáticamente con herramientas informáticas, está explícitamente prohibido, independientemente del motivo alegado.
- el departamento comprobará la copia entre todas las entregas de todos los grupos que intervienen en la convocatoria y considerará copia cuando aparezca esa condición en cualquiera de las entregas analizadas.
- el departamento realizará finalmente un análisis global de posibles casos de copia, incluyendo la inspección visual de implementaciones de los grupos cuyo código presente un mayor índice de coincidencias. Igualmente, se comprobarán algunas implementaciones elegidas al azar.